

ASRR: Action Sequence Residual Refinement for Frozen Vision-Language-Action Policy Adaptation

Anonymous ACL submission

Abstract

Multimodal signals such as language instructions, visual observations, robot states, and actions are central to modern robotic manipulation. Vision-language-action (VLA) models and action-sequence policies use these signals to generate temporally structured action sequences for robot control, but the resulting sequences are often useful yet imperfect: they may capture the coarse task intent while still reducing rollout success through local action errors or insufficient training. We propose *Action Sequence Residual Refinement (ASRR)*, an offline adaptation method that freezes a base policy and learns a lightweight *Action Sequence Refiner* to edit the predicted action sequence in the policy-native action space. Instead of updating the base model, ASRR acts as a plug-in residual correction layer that adds a bounded and optionally masked refinement to the base action proposal. We evaluate ASRR across mainstream action-sequence policies and VLA methods under different base-quality regimes. Adding the refiner substantially improves imitation-style policies, for example increasing ACT insertion success by +21.7 percentage points under temporal evaluation. Among VLA policies, ASRR is most effective on $\pi_{0.5}$, improving the average LIBERO suite success by +13.2 percentage points, while also producing net-positive improvements on other tested VLA methods. These results support viewing action sequences as editable policy proposals and provide a low-cost offline strategy for adapting frozen robot policies.

1 Introduction

Language-conditioned robot control has developed rapidly, with current methods increasingly connecting visual observations and natural-language instructions to executable manipulation (Ahn et al., 2022; Driess et al., 2023). Vision-language-action (VLA) systems, such as RT-2,

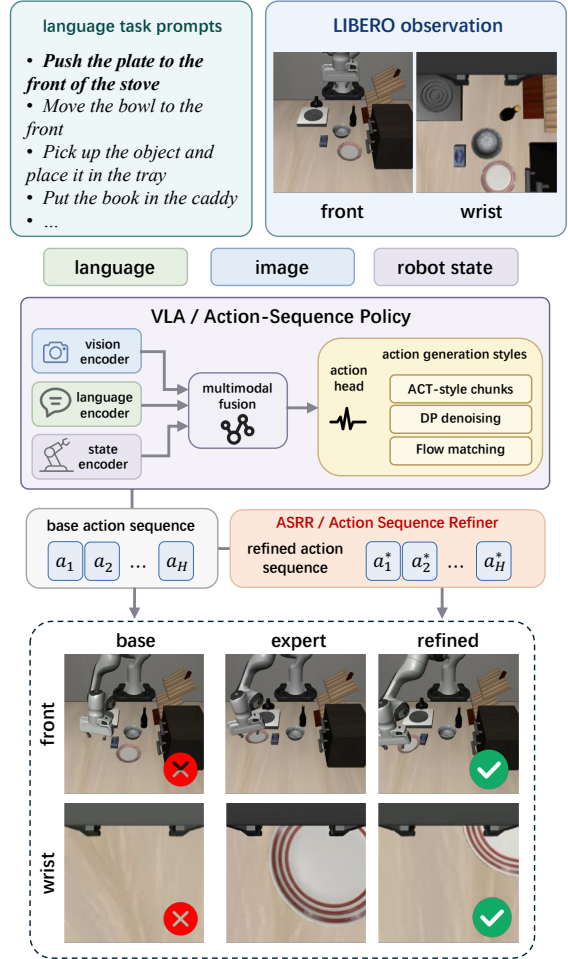


Figure 1: Problem setting. Frozen action-sequence policies often produce useful but imperfect action chunks; ASRR studies whether those chunks can be refined offline without updating the base policy.

OpenVLA, and π_0 , address this problem by mapping image, language, and robot state to actions (Brohan et al., 2023; Kim et al., 2024; Black et al., 2024). As these models scale, however, adaptation becomes costly.

In practical experiments, trained or fine-tuned robot policies often remain imperfect, and repeatedly performing large-scale fine-tuning to im-

prove their behavior requires substantial computation and time (Houlsby et al., 2019; Hu et al., 2022; Kim et al., 2025). A useful robot policy may also need to adapt to a new manipulation distribution, a changed camera setup, or a new execution protocol, while preserving behaviors that already work.

A common structure across modern robot policies is that actions are generated as temporally structured sequences. Action Chunking with Transformers (ACT) predicts action chunks (Zhao et al., 2023), Diffusion Policy (DP) predicts action trajectories through denoising (Chi et al., 2025), and recent VLA policies increasingly expose action chunks, action heads, or flow-style action generation. For example, OpenVLA-OFT optimizes continuous action chunks and parallel decoding, while π_0 and SmolVLA use flow-style or chunked action generation for VLA control (Kim et al., 2025; Lipman et al., 2023; Black et al., 2024; Shukor et al., 2025). These predicted sequences are usually treated as outputs to execute. In many failures, the trajectory direction is broadly plausible, but the action sequence remains coarse or locally inaccurate; this reflects the known gap between supervised action learning and closed-loop rollout behavior (Ross et al., 2011; Mandlekar et al., 2021). This makes action-sequence refinement a natural adaptation target.

Existing correction strategies do not directly solve this setting. Full fine-tuning or parameter-efficient tuning changes the base policy and can still require substantial training effort (Houlsby et al., 2019; Hu et al., 2022; Kim et al., 2025). Dataset aggregation and residual reinforcement learning address sequential error or policy correction, while recent VLA reinforcement fine-tuning methods add corrective learning signals, but these approaches typically require interactive data, value learning, or policy updates rather than an offline edit of the policy’s own proposed future (Ross et al., 2011; Silver et al., 2018; Johannink et al., 2019; Chen et al., 2025; Huang et al., 2025). For large VLA models, this motivates further exploration of methods that are offline, cheap, compatible with action-sequence policies, and able to use the base policy’s action proposal as the object being corrected.

We propose *Action Sequence Residual Refinement* (ASRR), a post-hoc refinement strategy for

frozen action-sequence policies.¹ Given a base action chunk A_{base} , an *Action Sequence Refiner* R_θ predicts a residual sequence and executes

$$A_{\text{refined}} = A_{\text{base}} + \alpha M \odot R_\theta(A_{\text{base}}, z), \quad (1)$$

where z denotes optional policy context, α controls residual strength, and M masks timesteps or action dimensions. The base policy remains frozen; only the refiner is trained. This makes ASRR a sequence-editing method over a policy’s own action proposal, rather than a new policy-training recipe. The main contributions of this paper are summarized as follows:

- We introduce ASRR, an offline frozen-base correction strategy that treats predicted action chunks as editable intermediate representations and validates the effectiveness of offline action-sequence refinement.
- We evaluate ASRR across action-sequence policies and VLA-style policies, including ACT, DP, SmolVLA, OpenVLA-OFT, $\pi_{0.5}$, and Octo, and observe consistent improvements in the tested settings.
- We show that the Action Sequence Refiner is lightweight and fast to train: across the tested base policies, the largest refiner uses less than 1.2% of the base policy’s parameters, while leaving the base policy unchanged.

2 Related Work

Language grounding and VLA policies. Language-conditioned manipulation connects instruction understanding with visual grounding and continuous robot control. SayCan grounds language-model plans in robotic affordances (Ahn et al., 2022), PaLM-E studies embodied multi-modal language modeling (Driess et al., 2023), RT-1 and RT-2 study large-scale transformer policies for real-world control (Brohan et al., 2022, 2023), and OpenVLA provides an open VLA model for robot control (Kim et al., 2024). Recent systems further broaden this interface: Octo supports language- or goal-conditioned generalist policies (Octo Model Team et al., 2024), π_0 uses flow-style action generation (Lipman et al., 2023; Black et al., 2024), $\pi_{0.5}$ targets open-world generalization (Black et al., 2025), and SmolVLA

¹Project page: <https://anonymous-0525.github.io/ASRR/>

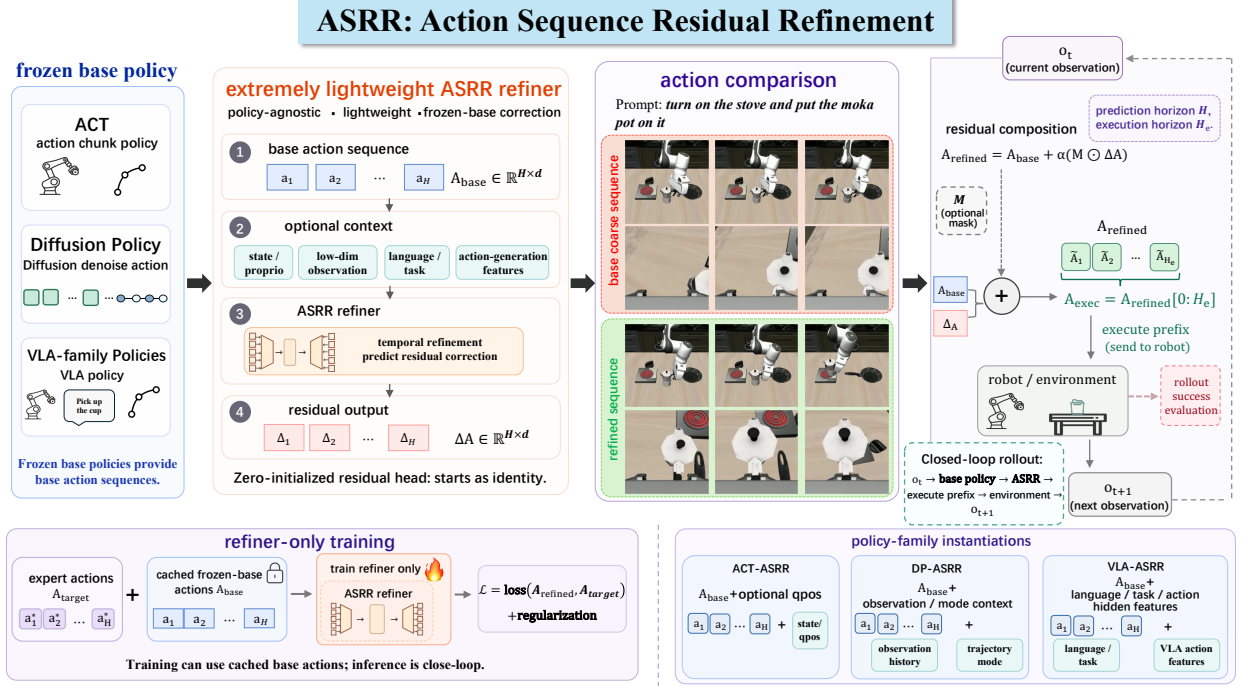


Figure 2: Overview of ASRR. A frozen base policy proposes an action sequence; an Action Sequence Refiner predicts a bounded residual over that proposal; the composed sequence is executed under the policy family’s execution protocol.

focuses on efficient chunked VLA deployment (Shukor et al., 2025). ASRR does not train a new VLA; it asks whether frozen VLA-style action sequences can be refined after prediction.

Action-sequence robot policies. Many robot policies predict an action horizon rather than a single action. ACT introduced action chunking and temporal ensembling for fine-grained bimanual manipulation (Zhao et al., 2023); DP learns visuomotor action trajectories with conditional denoising (Chi et al., 2025); Behavior Transformer-style models represent behavior as multi-modal sequence generation over latent or tokenized actions (Shafiullah et al., 2022; Lee et al., 2024); and OpenVLA-OFT optimizes VLA fine-tuning around continuous action chunks, parallel decoding, and action regression (Kim et al., 2025). These methods differ in architecture, but all expose a temporally structured action proposal that ASRR can edit.

Fine-tuning and residual correction. The standard way to adapt a policy is to continue training it, possibly with parameter-efficient mechanisms such as adapters or LoRA (Houlsby et al., 2019; Hu et al., 2022). Imitation-learning work such as DAGger explicitly addresses distribution

shift through learner-induced data collection (Ross et al., 2011), while residual policy learning adds corrective behavior on top of a base controller or policy (Silver et al., 2018; Johannink et al., 2019). Our setting is different: the residual is not a new state-to-action controller, and it is not a low-rank update inside the model. It is an offline output-space refinement operator over the predicted action sequence. The base policy is frozen, the refiner is trained from cached actions or demonstrations, and the same equation applies to ACT, DP, and VLA-style action generators.

3 Action Sequence Residual Refinement

3.1 Frozen Action-Sequence Proposals

Let π_b denote a frozen base policy. At control time t , the policy receives observation o_t and optional task or language condition c_t , and proposes an action sequence

$$A_{b,t} = \pi_b(o_t, c_t) = [a_{b,t}^0, \dots, a_{b,t}^{H_p-1}] \in \mathbb{R}^{H_p \times d_a}. \quad (2)$$

Here H_p is the prediction horizon and d_a is the action dimension in the policy-native action space. ASRR does not update the base policy. It introduces a trainable Action Sequence Refiner R_θ that edits the proposed sequence directly in that native

action representation. The intended regime is a competent but imperfect base: if $A_{b,t}$ has no useful task structure, the refiner would have to learn a new policy; if the base is already near-optimal, the refinement must be conservative.

3.2 Action Sequence Refiner

The refiner treats the base action chunk as a sequence of editable action tokens. Each timestep is embedded with a small action encoder and a positional embedding:

$$x_h = \phi_a(a_{b,t}^h) + p_h, \quad h = 0, \dots, H_p - 1. \quad (3)$$

ASRR separates the residual form from the source of context. We use two optional context types: global context z_t , shared across action timesteps, and per-step action-generation context $g_{t,h}$, aligned with the h -th action in the base chunk. The token input is

$$u_h = x_h + \phi_z(z_t) + \phi_g(g_{t,h}). \quad (4)$$

If a context source is unavailable, its embedding is set to zero. Thus ACT can use no context or qpos, DP can use low-dimensional observation or trajectory-mode context, and VLA policies can expose action-generation hidden states without changing the ASRR interface.

The fused tokens are processed by a lightweight temporal encoder,

$$H_{0:H_p-1} = \text{Enc}_\theta(u_{0:H_p-1}), \quad (5)$$

and a residual head predicts

$$\Delta A_t = \psi_\theta(H_{0:H_p-1}) \in \mathbb{R}^{H_p \times d_a}. \quad (6)$$

The final residual head is initialized to zero, so $\Delta A_t = 0$ at initialization and the composed policy starts exactly as the frozen base policy.

3.3 Masked Residual Composition

The refined action sequence is

$$\tilde{A}_t = A_{b,t} + \alpha(M \odot \Delta A_t), \quad (7)$$

where α is a scalar residual strength, M is an action-dimension or timestep mask, and $\odot$ denotes element-wise multiplication. In simple ACT and DP settings, M can be all ones. In VLA settings, M commonly freezes gripper or padding dimensions so that ASRR edits only the action coordinates for which a small residual correction is meaningful.

When conservative correction is needed, the residual is bounded by a policy-native trust region:

$$\Delta A_t = \delta_{\max} \odot \tanh(\psi_\theta(H_{0:H_p-1})), \quad (8)$$

where $\delta_{\max}$ can be a scalar or per-dimension bound. ACT and DP main settings allow dense residual heads, while VLA settings use bounded residuals and action masks to reduce negative transfer.

3.4 Policy-Native Action Spaces

ASRR applies the residual before any policy-specific output transform. This is important for VLA-style policies whose action tensors may be normalized, padded, or represented in an internal action space. For OpenVLA-OFT, refinement is applied to the normalized 8-step action chunk before unnormalization. For $\pi_{0.5}$, refinement is applied to the normalized internal action tensor; the pose dimensions are editable while the gripper and padded dimensions are frozen by M , and the existing output transform maps the refined tensor back to simulator actions. This avoids mixing residual learning with unnormalization, slicing, or padding conventions.

3.5 Offline Sequence-Editing Objective

Given a target sequence A_t^* , ASRR supervises the composed sequence in the base policy’s native action space. Masks choose editable coordinates, while horizon weights choose whether supervision covers the full chunk or only the executed prefix:

$$\mathcal{L}_{\text{sup}} = \frac{1}{Z_w} \sum_{h=0}^{H_p-1} w_h \ell(\tilde{a}_t^h, a_t^{*,h}),$$

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}} [\mathcal{L}_{\text{sup}}] + \lambda_\Delta \mathcal{L}_\Delta + \lambda_s \mathcal{L}_{\text{smooth}}, \quad (9)$$

where $Z_w = \sum_h w_h$, $\mathcal{L}_\Delta = \|M \odot \Delta A_t\|_F^2$, and $\mathcal{L}_{\text{smooth}} = \sum_{h=1}^{H_p-1} \|M \odot (\Delta a_t^h - \Delta a_t^{h-1})\|_2^2$. The first term trains the refined sequence to match demonstrations, cached targets, or teacher actions; the latter terms keep residuals small and temporally smooth. Some implementations add an L_1 residual penalty, which we treat as part of the residual regularizer. In our experiments, ACT uses an L1 sequence loss, while DP and VLA variants use MSE or masked MSE in the appropriate action space.

Family	Native proposal	Refiner context	Main constraint	Evaluation note
ACT (Zhao et al., 2023)	action chunk	action-only or qpos	dense residual	chunk / temporal eval
DP (Chi et al., 2025)	executable action horizon	low-dim obs and/or mode embedding	dense residual	fixed action horizon
OpenVLA-OFT (Kim et al., 2025)	normalized 7D LIBERO chunk	action hidden, proprio, task id	bounded / gated / low-rank; gripper frozen	official chunk eval
Octo* (Octo Model Team et al., 2024)	action chunk	action, readout, task features	bounded MLP residual	10k FT base; Object uses 30k; horizon 4
$\pi_{0.5}^*$ (Black et al., 2024, 2025)	normalized internal 32D action tensor	final denoising action features	bounded pose residual; gripper/padding frozen	10k FT base; output transform after refinement
SmolVLA (Shukor et al., 2025)	VLA action chunk	action-generation hidden states	bounded gated residual; gripper frozen	prefix / first-action aligned

Table 1: Main ASRR instantiations used in the experiments. The base is frozen in all cases; rows differ in native action space, exposed context, mask or bound, and execution protocol. * denotes 10k fine-tuned VLA bases except Octo Object, where we use the 30k checkpoint because the matched 10k Object base is unusually weak. Full settings are provided in Appendix Table 3.

3.6 Execution-Aware Refinement

The horizon weights w_h align training with execution. This yields three common choices:

$$w_h = 1 \quad \text{full-horizon training,} \quad (10)$$

$$w_h = \mathbf{1}[h < H_e] \quad \text{executed-prefix training,} \quad (11)$$

$$w_h = \mathbf{1}[h = 0] \quad \text{first-action training,} \quad (12)$$

where H_e is the number of actions executed before the next policy query. At inference, ASRR refines the base proposal and executes only the protocol-specific prefix:

$$\begin{aligned} \pi_{\text{ASRR}}(o_t, c_t) &= \text{Exec}_{H_e}(\tilde{A}_t), \\ \Delta A_t &= R_\theta(A_{b,t}, z_t, g_{t,0:H_p-1}), \\ \tilde{A}_t &= A_{b,t} + \alpha(M \odot \Delta A_t). \end{aligned} \quad (13)$$

The policy is then queried again after the executed prefix. For ACT and DP, the supervised and executed horizons are naturally aligned with their evaluation protocols. For VLA policies that predict long chunks but execute only a short prefix, the choice of w_h is part of the method rather than a minor implementation detail.

4 Experiments

4.1 Experimental Setup

Tasks and data. The experiments cover action-sequence imitation policies and VLA policies. ACT is evaluated in the official simulated manipulation setup on insertion and transfer cube tasks. DP uses the standard demonstration benchmarks used by Diffusion Policy and Robomimic, with Square and Transport as the main tasks and PushT and Tool-Hang as additional DP evidence (Chi et al., 2025; Mandlekar et al., 2021). VLA

experiments use LIBERO Spatial, Object, Goal, and LIBERO-10 suites (Liu et al., 2023). Policy-specific datasets, checkpoints, context inputs, and alpha-selection details are summarized in Appendix Table 3.

Evaluation protocol. All experiments freeze the base policy and train only the Action Sequence Refiner. The evaluation unit is a rollout episode, and the primary metric is task success. We report absolute gain over the frozen base, trainable refiner parameters, and residual strength where applicable. For paired evaluations with matched initial states, we also report rescue and harm: rescue counts base-failed episodes made successful by ASRR, while harm counts base-success episodes made unsuccessful by ASRR.

Base-quality coverage. Several settings involve candidate refiners or residual-strength sweeps, so selection and final evaluation are reported explicitly. ACT results use six training seeds and 50 rollouts per evaluation mode. DP main results aggregate over five undertrained checkpoints per task under the primary sweep, with fresh-seed robustness reported separately. For VLA policies, we use both strong official or public fine-tuned checkpoints and weaker or undertrained bases to study how proposal quality affects residual refinement. ASRR never updates the base weights.

4.2 Base Policies and ASRR Instantiations

ACT emits action chunks that can be refined with either action-only or qpos-conditioned ASRR. DP emits executable action horizons from a generative policy; we therefore test action-only, observation-conditioned, mode-conditioned, and observation+mode refiners. VLA policies expose

continuous action chunks or action-generation features; ASRR operates in the policy-native action space and uses bounded or masked residuals when needed, such as freezing gripper or padding dimensions. Table 1 summarizes the main instantiations, while Appendix Table 3 records the detailed experimental settings.

4.3 Main Results

Action-sequence imitation policies. ACT provides the cleanest positive case for ASRR: the action-only refiner yields large gains on insertion and smaller but consistent gains on the already-strong transfer cube base. DP shows that ASRR transfers to generative action-horizon policies, but with stronger dependence on context; observation and mode features outperform purely action-only refinement. Additional DP evaluations on PushT and Tool-Hang are included as broader evidence in Table 2, with detailed sweeps in Appendix Tables 8–10.

VLA policies. VLA policies are the high-heterogeneity setting for ASRR because they combine language, vision, state, and multi-task action-generation modules. Table 2 reports completed OpenVLA-OFT, Octo, $\pi_{0.5}$, and SmolVLA evaluations. OpenVLA-OFT is a strong official LIBERO fine-tuned base, so the headroom is small; nevertheless, its suite-specific refiners are net positive on all four suites, improving the aggregate result from 1916/2000 to 1929/2000 with 36 rescues and 23 harms.

Octo and $\pi_{0.5}$ provide broader VLA-family evidence on 10k fine-tuned checkpoints. The larger $\pi_{0.5}$ gains suggest that ASRR is especially effective when the base proposal is useful but still locally under-refined. Octo shows the complementary boundary case: Spatial, Goal, and LIBERO-10 improve with readout+action+task context, but Object has a very weak base and gains only slightly with an action-only refiner. SmolVLA also improves on all four suites with the simpler gripper-frozen refiner, but with larger rescue and harm counts, reinforcing the need to analyze VLA refinement as a paired intervention rather than only through aggregate success.

4.4 Analysis and Ablations

The current ablations address four design questions. First, context is policy-dependent: ACT action-only refinement is already strong, while

DP improves when observation and mode context are supplied, suggesting that context helps disambiguate generative action proposals. Second, the useful refinement horizon depends on execution mode: ACT H30 is strongest for chunk evaluation, while H80 is strongest for temporal evaluation. Third, residual strength controls a rescue-harm tradeoff, illustrated by the $\pi_{0.5}$ alpha sweep, where the best residual scale is suite-dependent: Spatial and Goal peak at larger alphas, LIBERO-10 peaks at 2.0, and Object saturates early. Fourth, DP refiner checkpoints show that later or best-loss checkpoints are not uniformly superior, so rollout selection remains necessary.

Figure 3–Figure 6 show the completed ablations used in the main analysis. The main text reports the newer net-positive SmolVLA simple-refiner results and omits earlier static-refiner diagnostics.

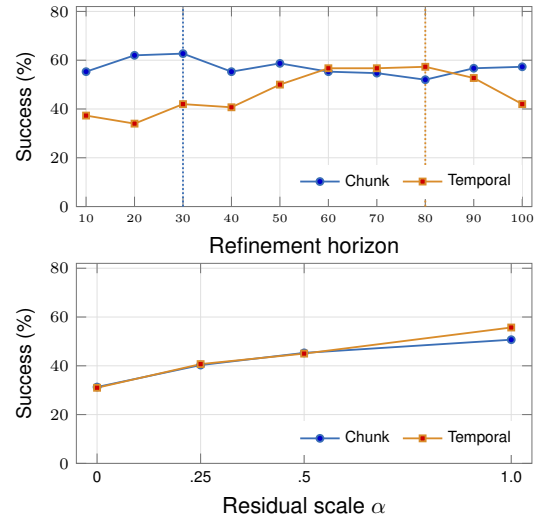


Figure 3: ACT horizon and residual-strength ablations. H30 favors chunk evaluation, while H80 and $\alpha = 1.0$ support the temporal setting.

5 Discussion

Action chunks as repairable policy representations. ASRR treats a predicted action chunk as a structured proposal, not only as a command to execute. The ACT results support this representation-level view: an action-only refiner can correct local pose, timing, and contact errors without re-solving perception or updating the base model. This suggests that competent base policies often expose enough temporal intent in the action sequence itself for local repair.

Context as disambiguation rather than extra capacity. Context is useful when the base pro-

Family	Method	Task / suite	Base (%)	Refined (%)	Δ (pp)	Rescue	Harm	Params: base / refiner
Policy	ACT	Insertion (300)	34.0	55.7	+21.7	+90	-25	83.92M / 159.5k
		Transfer cube (300)	89.3	95.3	+6.0	+30	-12	83.92M / 159.5k
	DP	Square (250)	81.6	92.0	+10.4	+38	-12	65.88M / 681.7k
		Transport (250)	76.0	84.0	+8.0	+39	-19	66.92M / 709.1k
		PushT (300)	79.1	82.0	+3.0	+129	-101	65.78M / 673.8k
		Tool-Hang (200)	66.0	79.5	+13.5	+48	-21	66.74M / 697.1k
VLA	OpenVLA-OFT	LIBERO-Spatial (500)	92.8	93.0	+0.2	+10	-9	7.54B / 7.66M
		LIBERO-Object (500)	98.8	99.4	+0.6	+5	-2	7.54B / 7.66M
		LIBERO-Goal (500)	96.4	97.4	+1.0	+10	-5	7.54B / 7.66M
		LIBERO-10 (500)	95.2	96.0	+0.8	+11	-7	7.54B / 7.66M
	Octo*	LIBERO-Spatial (500)	84.6	87.0	+2.4	+32	-20	202.46M / 707k
		LIBERO-Object (500)	55.0	56.8	+1.8	+30	-21	202.46M / 80k
		LIBERO-Goal (500)	88.2	90.8	+2.6	+40	-27	202.46M / 707k
		LIBERO-10 (500)	51.0	61.6	+10.6	+98	-45	202.46M / 707k
	$\pi_{0.5}$ *	LIBERO-Spatial (500)	84.4	96.0	+11.6	+74	-16	3.3B / 4.0M
		LIBERO-Object (500)	88.4	99.8	+11.4	+58	-1	3.3B / 4.0M
		LIBERO-Goal (500)	70.4	84.2	+13.8	+83	-14	3.3B / 4.0M
		LIBERO-10 (500)	68.6	84.6	+16.0	+115	-35	3.3B / 4.0M
	SmoIVLA	LIBERO-Spatial (500)	66.4	68.6	+2.2	+102	-91	450.05M / 5.18M
		LIBERO-Object (500)	80.8	81.8	+1.0	+55	-50	450.05M / 5.18M
		LIBERO-Goal (500)	73.0	77.4	+4.4	+72	-50	450.05M / 5.18M
		LIBERO-10 (500)	33.4	35.0	+1.6	+77	-69	450.05M / 5.18M

Table 2: Main ASRR result table. Base and refined columns are percentages; the task/suite column gives the number of evaluation episodes in parentheses. Δ is in percentage points for success tasks and score points for PushT, where the percentage denotes normalized coverage rather than binary success. Rescue and harm use paired rollout indices and initial seeds when available; for PushT, these columns count refined rollouts with higher or lower coverage score than the base rollout. Greener cells indicate more rescued or improved base rollouts; redder cells indicate more harmed or lower-scoring rollouts. * denotes 10k fine-tuned VLA checkpoints; Octo Object uses a 30k checkpoint because the otherwise matched 10k object base was unusually weak under the same protocol. Other rows use the standard checkpoint or configuration for that policy family. Full training, checkpoint, alpha, and evaluation details are listed in the appendix.

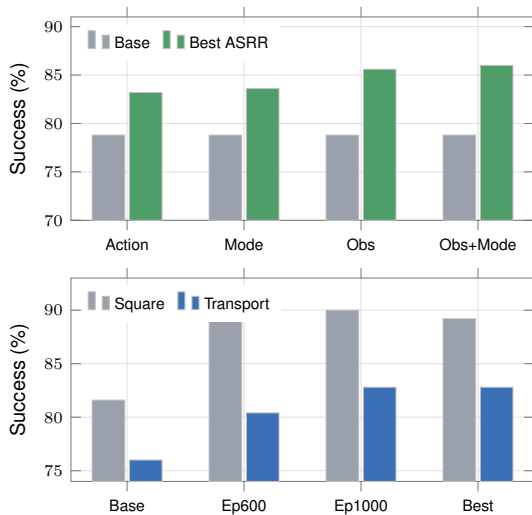


Figure 4: DP context and refiner-checkpoint ablations. Observation/mode context performs best, and rollout success is not monotonic across refiner checkpoints.

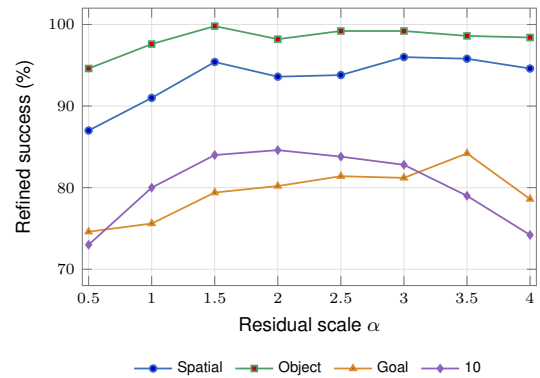


Figure 5: $\pi_{0.5}$ residual-strength sweep over 500 paired LIBERO rollouts per suite. The best residual scale is suite-dependent: Spatial and Goal peak at larger alphas, while LIBERO-10 peaks at 2.0.

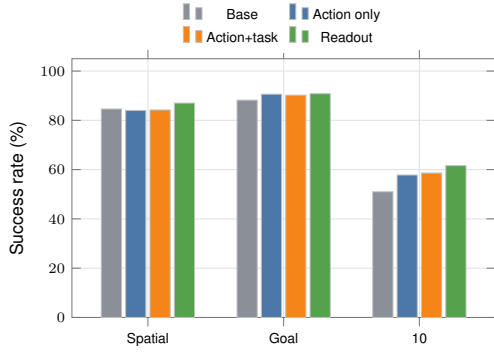


Figure 6: Octo structure ablation over 500 rollouts per suite. Readout denotes readout+action+task; Object is omitted because its weak base prefers a small action-only correction.

posals is ambiguous. ACT-style chunks can often be edited from the action sequence alone, while DP and VLA policies benefit from observation, mode, task, or action-generation features that identify which trajectory proposal is being refined. Thus, context in ASRR is best understood as disambiguation rather than as generic extra capacity.

VLA refinement as paired intervention. For VLA policies, refinement is an intervention on an existing behavior. Aggregate success can hide whether gains come from selective rescue or from broad perturbation that also harms successful base rollouts. Paired rescue and harm counts therefore provide a more informative evaluation than success rate alone, especially for strong or high-variance VLA bases.

Execution alignment matters for sequence refinement. ASRR refines a predicted sequence, but closed-loop control may execute only a prefix before the next observation. If training emphasizes timesteps that are rarely executed, the learned residual can be misaligned with the deployed intervention. Execution-aware weighting is therefore a mechanism for matching supervised sequence editing to rollout behavior, not a minor implementation detail.

Intended regime of ASRR. ASRR is best understood as a conservative specialization layer for competent but imperfect bases. It is most useful when the base trajectory is directionally meaningful but locally inaccurate in pose alignment, contact timing, gripper coordination, or smoothness. Residual scale, masking, context, and execution horizon define how aggressively this proposal is edited.

6 Conclusion

We studied Action Sequence Residual Refinement as a correction method for the action-sequence outputs of robot policies and language-conditioned VLA models. First, ASRR treats the predicted action sequence as an editable intermediate representation: a frozen base policy proposes an action chunk, a lightweight refiner predicts a residual correction, and the corrected sequence is executed under the original policy protocol. Second, across action-sequence imitation policies and VLA-style models, the results indicate that this post-hoc structure can improve policies with different architectures and training stages while keeping the base model unchanged. Finally, because the refiner is small, fast to train, and easy to attach to policies that expose action chunks or action-generation features, ASRR provides a practical framework for studying how action proposals from existing robot policies can be reused, corrected, and further specialized without repeatedly fine-tuning the full policy.

7 Limitations

ASRR assumes that the base policy exposes an action sequence or can be wrapped as a sequence-producing policy. Its behavior is affected by the number of predicted and executed action steps: short execution horizons make ASRR closer to local residual correction, while longer chunks provide more temporal structure. The method also depends on base-policy competence: if the base lacks task structure, the refiner can only provide limited recovery.

The reported VLA results use frozen checkpoints, suite-specific refiners, and residual-strength selection on validation or pilot rollouts. These settings study post-hoc adaptation, but they do not imply that one universal refiner or alpha will improve every VLA, suite, or checkpoint. Paired rescue-harm reporting is therefore necessary because small aggregate gains can hide both rescued failures and harmed successes.

Most evaluations are simulation-based, and deployment distributions may differ in camera placement, object initialization, dynamics, and latency. Online data collection, value-based gating, and real-robot validation remain important extensions.

Ethics Statement

This work studies robotic policy adaptation using simulation and offline demonstration data. It does not involve human-subject data or private personal information. Action Sequence Refiners can change the behavior of a base policy, so deployment should include explicit checks for correction magnitude, workspace safety, and gripper events.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, et al. 2022. Do As I Can, Not As I Say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*.

Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, et al. 2025. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *9th Annual Conference on Robot Learning*.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, et al. 2024. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, et al. 2023. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, et al. 2022. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*.

Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. 2025. Con-RFT: A reinforced fine-tuning method for VLA models via consistency policy. *arXiv preprint arXiv:2502.05450*.

Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2025. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*.

Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, et al. 2023. PaLM-e: An embodied multi-modal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR.

Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Dongchi Huang, Zhirui Fang, Tianle Zhang, Yihang Li, Lin Zhao, and Chunhe Xia. 2025. CO-RFT: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning. *arXiv preprint arXiv:2508.02219*.

Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. 2019. Residual reinforcement learning for robot control. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 6023–6029.

Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*.

Seungjae Lee, Yibin Wang, Haritheja Etukuru, H. Jin Kim, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. 2024. [Behavior generation with latent actions](#). In *Multi-modal Foundation Model meets Embodied AI Workshop @ ICML2024*.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. 2023. [Flow matching for generative modeling](#). In *The Eleventh International Conference on Learning Representations*.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. [LIBERO: Benchmarking knowledge transfer for lifelong robot learning](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 44776–44791. Curran Associates, Inc.

Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. 2021. [What matters in learning from offline human demonstrations for robot manipulation](#). In *5th Annual Conference on Robot Learning*.

Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. 2024. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*.

Stephane Ross, Geoffrey Gordon, and Drew Bagnell. 2011. [A reduction of imitation learning and structured prediction to no-regret online learning](#). In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, Fort Lauderdale, FL, USA. PMLR.

Nur Muhammad Shafiullah, Zichen Cui, Ariuntuya (Arty) Altanzaya, and Lerrel Pinto. 2022. [Behavior transformers: Cloning \$k\$ modes with one stone](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 22955–22968. Curran Associates, Inc.

Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. 2025. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*.

Tom Silver, Kelsey Allen, Joshua B. Tenenbaum, and Leslie Pack Kaelbling. 2018. Residual policy learning. *arXiv preprint arXiv:1812.06298*.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. [Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware](#). In *Pro-*

ceedings of Robotics: Science and Systems, Daegu, Republic of Korea.

681
682

A Appendix Organization

Appendix Table 3 gives the compact experiment settings used by the main text. The following sections then provide additional ACT, DP, and VLA tables, plus parameter-efficiency summaries.

Data and artifacts. We use public datasets, benchmarks, and checkpoints only for research evaluation and do not redistribute original artifacts.

B Policy-Specific Experimental Settings

Policy	Tasks / data	Base policy	ASRR context and mask	Selection and evaluation
ACT	Official simulated insertion and transfer cube tasks	Three-camera ACT, 2000 epochs, six seeds	Action-only H80 and qpos H80; dense residual over the action chunk	501 refiner epochs; 50 rollouts per seed; chunk and temporal evaluation; H30/H40/H80 horizon diagnostics
DP	Robomimic-style Square and Transport; PushT and Tool-Hang as auxiliary tasks	Frozen DP checkpoints e60/e80/e100/e300/e500 for main tasks	Action-only, obs-add, mode, and obs+mode refiners over executable action horizons	1000 refiner epochs; best/epoch0600/epoch1000 checkpoints; $\alpha \in \{0.25, 0.5, 1.0\}$; main seed plus fresh-seed robustness
OpenVLA-OFT	LIBERO Spatial, Object, Goal, 10	Official LIBERO fine-tuned checkpoint	Mean-pooled action hidden context, proprio, suite task id; normalized 7D action; gripper frozen	1000 cache samples/task; 12 candidates/suite from four structures and three bounds; full 500-episode paired evaluation; route results excluded
Octo*	LIBERO Spatial, Object, Goal, 10	Suite-specific 10k fine-tuned checkpoints; Object uses the 30k checkpoint because the matched 10k Object base is unusually weak	Action/readout/task feature variants; bounded MLP residual; best variant differs by suite	5000 refiner steps; $\alpha \in \{0.25, 0.5, 0.75, 1.0, 1.25\}$; 500 rollouts/suite under corrected image/action protocol
$\pi_{0.5}^*$	LIBERO Spatial, Object, Goal, 10	Step-10000 fine-tuned checkpoint, 3.3B params	Final denoising action features; normalized internal 32D tensor; pose-only residual with gripper/padding frozen, 4.0M params	$\alpha = 0.5\text{--}4.0$ in 0.5 steps; 500 paired rollouts/suite; best alpha selected per suite
SmolVLA	LIBERO Spatial, Object, Goal, 10	Frozen SmolVLA checkpoint under simple ASRR protocol	Action-generation hidden states; bounded gated residual; gripper frozen	500 paired rollouts per suite; report delta magnitude, gate mean, rescue, and harm

Table 3: Detailed experimental settings referenced by Section 4. * denotes 10k fine-tuned VLA bases except Octo Object, where we use the 30k checkpoint because the matched 10k Object base is unusually weak. Full per-method sweeps and timing details are reported in the following appendix tables.

C Additional ACT Tables and Diagnostics

H	Chunk mean	Chunk std	Chunk min	Chunk max	Temp. mean	Temp. std	Temp. min	Temp. max
10	0.553	0.147	0.380	0.740	0.373	0.189	0.220	0.640
20	0.620	0.107	0.480	0.740	0.340	0.201	0.160	0.620
30	0.627	0.090	0.500	0.700	0.420	0.204	0.220	0.700
40	0.553	0.109	0.400	0.640	0.407	0.165	0.280	0.640
50	0.587	0.090	0.460	0.660	0.500	0.157	0.360	0.720
60	0.553	0.084	0.440	0.640	0.567	0.106	0.440	0.700
70	0.547	0.133	0.360	0.660	0.567	0.077	0.460	0.640
80	0.520	0.071	0.460	0.620	0.573	0.100	0.440	0.680
90	0.567	0.084	0.480	0.680	0.527	0.084	0.440	0.640
100	0.573	0.025	0.540	0.600	0.420	0.082	0.320	0.520

Table 4: ACT H10–H100 action-only horizon sweep on insertion using seeds 0–2. H30, H40, and H80 were subsequently expanded to six seeds and are summarized in the main text.

Checkpoint	Chunk mean	Chunk std	Chunk min	Chunk max	Temp. mean	Temp. std	Temp. min	Temp. max
epoch100	0.537	0.082	0.400	0.680	0.513	0.060	0.440	0.600
epoch300	0.500	0.076	0.400	0.640	0.583	0.075	0.480	0.680
epoch500	0.487	0.083	0.380	0.620	0.557	0.080	0.440	0.680
last	0.487	0.100	0.360	0.660	0.560	0.092	0.420	0.680
best	0.507	0.074	0.400	0.620	0.557	0.097	0.420	0.680

Table 5: ACT checkpoint diagnostics for insertion / top-left-right / action-only / H80. Useful correction is not restricted to a single fragile checkpoint.

Alpha	Chunk mean	Chunk std	Chunk min	Chunk max	Temp. mean	Temp. std	Temp. min	Temp. max
0.0	0.313	0.154	0.120	0.560	0.310	0.205	0.040	0.560
0.25	0.403	0.155	0.200	0.700	0.407	0.221	0.080	0.760
0.5	0.453	0.148	0.280	0.760	0.450	0.166	0.240	0.740
1.0	0.507	0.074	0.400	0.620	0.557	0.097	0.420	0.680

Table 6: ACT alpha sweep for insertion / top-left-right / action-only / H80. In this ACT setting, stronger residual scaling improves both chunk and temporal evaluation up to $\alpha = 1.0$.

Setting	Trainable params	Param ratio	Epochs	Train wall/seed	Chunk eval/seed	Temporal eval/seed
ACT three-camera base	83.92M	100%	2000	58–65 min	8:35–8:33	12:48–12:43
ASRR-action H30	146,702	0.17%	501	13:39	8:00	11:43
ASRR-action H80	159,502	0.19%	501	12:59	9:36	11:52
ASRR-qpos H80	229,134	0.27%	501	13:29	8:28	13:46

Table 7: ACT compute summary from the final timing logs. Evaluation uses 50 rollouts with episode saving disabled.

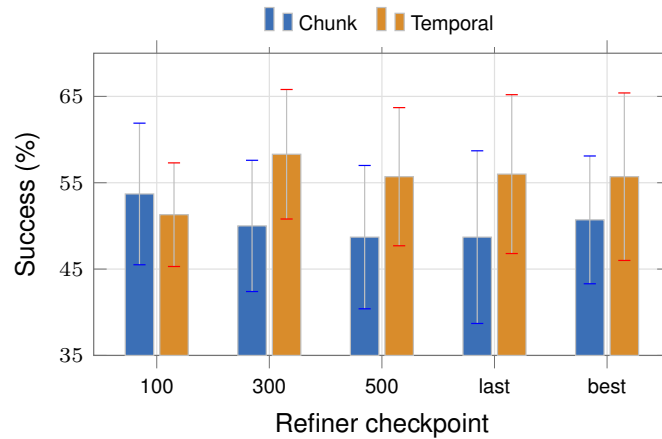


Figure 7: ACT refiner checkpoint diagnostic. Useful correction is not an artifact of a single refiner checkpoint; the main text reports the more compact horizon and residual-strength ablations.

D Additional Diffusion Policy Tables

690

Variant	Base mean	Best within variant	Gain
Action	0.788	0.832	+0.044
Mode	0.788	0.836	+0.048
Obs-add	0.788	0.856	+0.068
Obs+mode	0.788	0.860	+0.072

Table 8: DP context-wise aggregate over the ten main Square/Transport checkpoint settings. Each entry uses the best refiner checkpoint and alpha for that context on each base checkpoint.

Task family	Variant	Params	Cache min	Train min	Total min
Square	Action	71,687	3.24	2.80	6.13
Square	Obs-add	149,511	3.25	3.12	6.48
Square	Mode	681,735	3.26	4.55	7.92
Square	Obs+mode	681,735	3.24	4.59	7.94
Transport	Action	75,278	3.25	3.10	6.45
Transport	Obs-add	171,534	3.24	3.33	6.67
Transport	Mode	709,134	3.25	4.94	8.28
Transport	Obs+mode	709,134	3.25	4.95	8.29

Table 9: DP refiner timing and parameter summary for the completed main evaluation. Times are average minutes per refiner train/cache job from the final DP timing CSV after excluding smoke runs. All main refiners train for 1000 epochs.

Task	Base (%)	Best ASRR (%)	Gain (pp)	Best variant
PushT e20	47.9	47.3	-0.5	Action
PushT e40	82.5	87.5	+5.0	Action
PushT e60	89.1	92.6	+3.5	Mode
PushT e80	87.4	89.3	+1.9	Action
PushT e100	84.3	90.3	+6.0	Obs-add
PushT e500	83.2	85.1	+1.9	Obs-add
Tool-Hang e60	46.0	68.0	+22.0	Obs+mode
Tool-Hang e80	78.0	86.0	+8.0	Mode
Tool-Hang e100	66.0	78.0	+12.0	Mode
Tool-Hang e500	74.0	86.0	+12.0	Mode
PushT mean	79.1	82.0	+3.0	mixed
Tool-Hang mean	66.0	79.5	+13.5	mixed

Table 10: Completed DP appendix results. PushT shows mild average gains with one slight negative checkpoint, while Tool-Hang shows stronger gains. These results broaden the DP evidence but are not the main DP claim.

Robustness task	Base mean (%)	ASRR mean (%)	Gain (pp)
Square e60	79.3	72.7	-6.7
Square e80	83.3	91.3	+8.0
Square e100	82.7	84.7	+2.0
Square e500	85.3	90.0	+4.7
Transport e80	72.0	71.3	-0.7
Transport e100	70.7	70.0	-0.7
Transport e300	70.0	70.0	+0.0
Transport e500	74.0	80.7	+6.7
Square mean	82.7	84.7	+2.0
Transport mean	71.7	73.0	+1.3

Table 11: DP fresh-seed robustness re-evaluation. Selected refiners are reused without retraining on seeds 200000, 300000, and 400000. The coverage is incomplete: square e300 and transport e60 were omitted by the robustness script.

E Additional VLA Results and Efficiency

This appendix records the detailed settings and auxiliary results that support the compact main tables. All VLA results below use the same frozen-base ASRR protocol described in the main text; route-table or exploratory variants are excluded unless explicitly marked as diagnostics.

E.1 VLA Suite Results

Table 12 reports the complete $\pi_{0.5}$ step-10000 alpha sweep from $\alpha = 0.5$ to $\alpha = 4.0$. The best alpha is selected separately for each LIBERO suite, showing that residual strength is suite-dependent rather than globally monotonic.

Suite	α	Base (%)	Refined (%)	Gain (pp)	Rescue	Harm	Preserve
Spatial	0.5	84.4	87.0	+2.6	65	52	370
Spatial	1.0	84.4	91.0	+6.6	68	35	387
Spatial	1.5	84.4	95.4	+11.0	72	17	405
Spatial	2.0	84.4	93.6	+9.2	69	23	399
Spatial	2.5	84.4	93.8	+9.4	75	28	394
Spatial	3.0	84.4	96.0	+11.6	74	16	406
Spatial	3.5	84.4	95.8	+11.4	74	17	405
Spatial	4.0	84.4	94.6	+10.2	73	22	400
Object	0.5	88.4	94.6	+6.2	52	21	421
Object	1.0	88.4	97.6	+9.2	58	12	430
Object	1.5	88.4	99.8	+11.4	58	1	441
Object	2.0	88.4	98.2	+9.8	58	9	433
Object	2.5	88.4	99.2	+10.8	58	4	438
Object	3.0	88.4	99.2	+10.8	58	4	438
Object	3.5	88.4	98.6	+10.2	58	7	435
Object	4.0	88.4	98.4	+10.0	57	7	435
Goal	0.5	70.4	74.6	+4.2	62	41	311
Goal	1.0	70.4	75.6	+5.2	58	32	320
Goal	1.5	70.4	79.4	+9.0	70	25	327
Goal	2.0	70.4	80.2	+9.8	68	19	333
Goal	2.5	70.4	81.4	+11.0	68	13	339
Goal	3.0	70.4	81.2	+10.8	75	21	331
Goal	3.5	70.4	84.2	+13.8	83	14	338
Goal	4.0	70.4	78.6	+8.2	69	28	324
LIBERO-10	0.5	68.6	73.0	+4.4	94	72	271
LIBERO-10	1.0	68.6	80.0	+11.4	105	48	295
LIBERO-10	1.5	68.6	84.0	+15.4	115	38	305
LIBERO-10	2.0	68.6	84.6	+16.0	115	35	308
LIBERO-10	2.5	68.6	83.8	+15.2	115	39	304
LIBERO-10	3.0	68.6	82.8	+14.2	119	48	295
LIBERO-10	3.5	68.6	79.0	+10.4	101	49	294
LIBERO-10	4.0	68.6	74.2	+5.6	95	67	276

Table 12: $\pi_{0.5}$ step-10000 alpha sweep. Each row uses 500 paired LIBERO rollout episodes. The bold entry marks the best refined success within each suite. Rescue and harm are paired episode counts relative to the frozen base.

Suite	Base	Full adapter	Gain	Rescue	Harm	Net	Alpha	Adapter
LIBERO-Spatial	464/500 (92.8%)	465/500 (93.0%)	+1	10	9	+1	2.0	A0 lowrank, B0
LIBERO-Object	494/500 (98.8%)	497/500 (99.4%)	+3	5	2	+3	0.25	A1 gated, B0
LIBERO-Goal	482/500 (96.4%)	487/500 (97.4%)	+5	10	5	+5	1.0	A1 gated, B2
LIBERO-10	476/500 (95.2%)	480/500 (96.0%)	+4	11	7	+4	0.5	A2 direct, B0
Total	1916/2000 (95.8%)	1929/2000 (96.45%)	+13	36	23	+13	–	suite-specific

Table 13: OpenVLA-OFT suite-specific full adapter results. Each suite uses 10 tasks $\times$ 50 LIBERO rollout episodes with paired base/adapter initial states. This table reports only the full suite-specific adapters and excludes route-table results. B0 uses bounds $[0.04, 0.04, 0.04, 0.015, 0.015, 0.015, 0.0]$; B2 uses $[0.12, 0.12, 0.12, 0.050, 0.050, 0.050, 0.0]$.

Suite	Base	ASRR	Gain (pp)	Rescue	Harm	Preserve	Both fail	Behavior stats
LIBERO-Spatial	66.4	68.6	+2.2	102	91	241	66	$ \Delta_0 = 0.0265$, gate=0.839
LIBERO-Object	80.8	81.8	+1.0	55	50	354	41	$ \Delta_0 = 0.0253$, gate=0.802
LIBERO-Goal	73.0	77.4	+4.4	72	50	315	63	$ \Delta_0 = 0.0269$, gate=0.850
LIBERO-10	33.4	35.0	+1.6	77	69	98	256	$ \Delta_0 = 0.0259$, gate=0.798
Total	63.4	65.7	+2.3	306	260	1008	426	gripper residual = 0

Table 14: SmolVLA simple-ASRR results on all four LIBERO suites. Each suite uses 500 paired rollouts. This newer simple refiner is net positive on every suite and freezes gripper correction.

Suite	Base	Readout	Action-only	Action+task	Best refiner	α action	α action+task	α readout	Notes
LIBERO-Spatial	84.6	87.0	84.0	84.2	readout	0.25	0.50	0.50	readout+action+task best
LIBERO-Goal	88.2	90.8	90.6	90.2	readout	0.75	0.75	0.75	action-only close second
LIBERO-Object	55.0	56.6	56.8	55.8	action-only	0.25	0.25	0.10	30k checkpoint; action-only best
LIBERO-10	51.0	61.6	57.8	58.6	readout	0.75	1.25	0.50	largest Octo gain

Table 15: Octo four-suite structure ablation. All refiners train for 5000 steps. Spatial, Goal, and LIBERO-10 use 10k fine-tuned suite bases; Object uses the 30k checkpoint because the matched 10k Object base is unusually weak, consistent with the main experimental settings. Readout denotes the readout+action+task feature mode used in the main compact plot.